

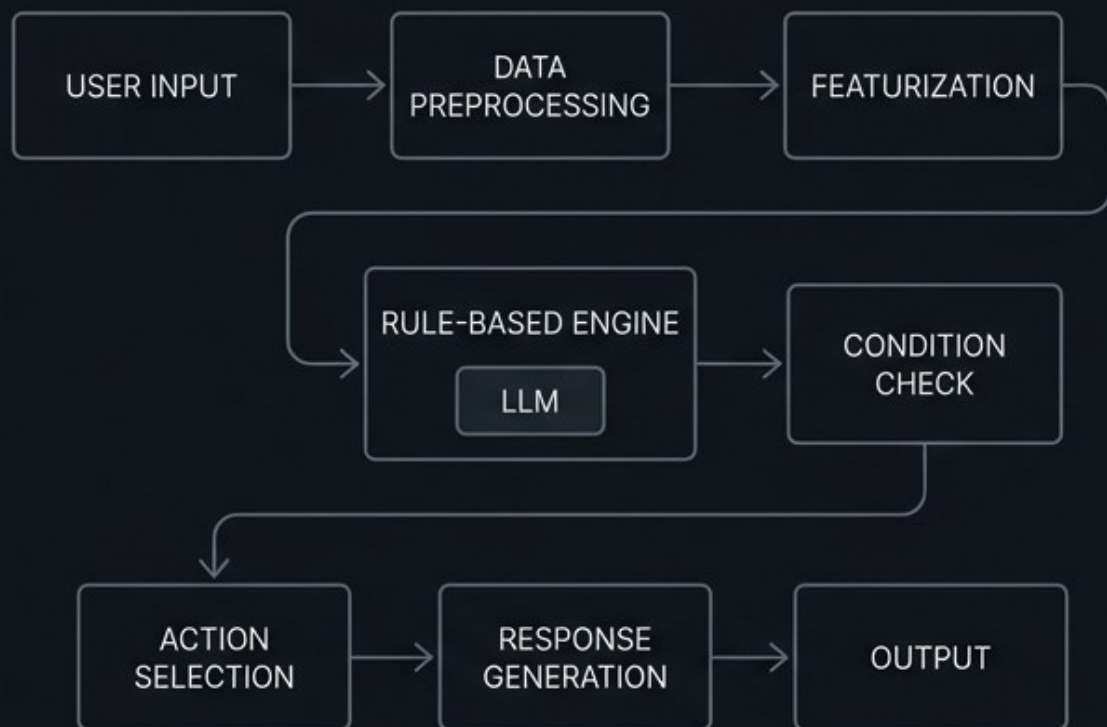
拆解 Claude Code 与 OpenClaw

开源 Kode 高性能 Agent Harness

- ShareAI Lab Xlu



大多数人在做的 Agent



真正做决策的是写流程图的人，不是模型。

Claude Code 完全相反



给模型搭一个完备的工作环境，然后说：“你自己来”。

bash

read_file

write_file

edit_file

有了这四个基础工具，探索、理解、修改、测试全能搞定。一台电脑该有的都有了。模型就是 Agent，代码只是 Harness。

PART 1. CLAUDE CODE

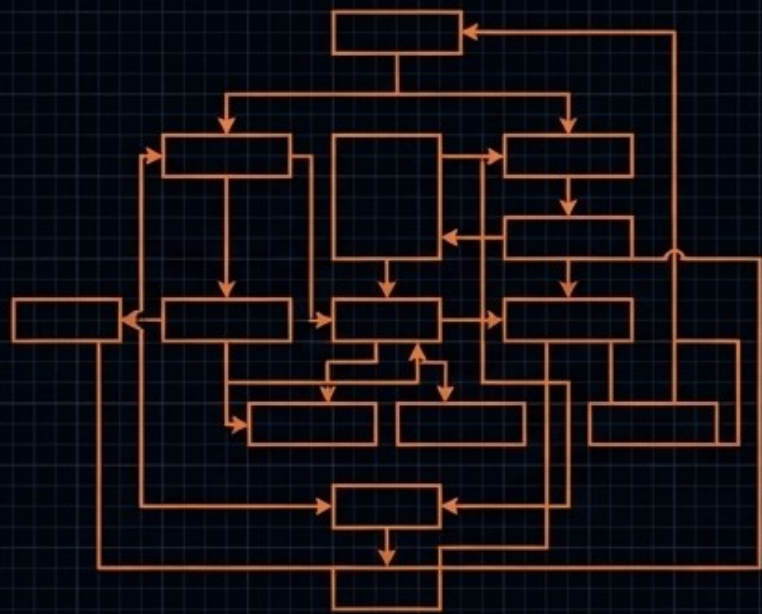
The Eternal Loop

- 放弃流程图。给模型 4 个基础工具。

循环永远不变。一切决策交还给模型。
循环永远不变。一切决策交还给模型。

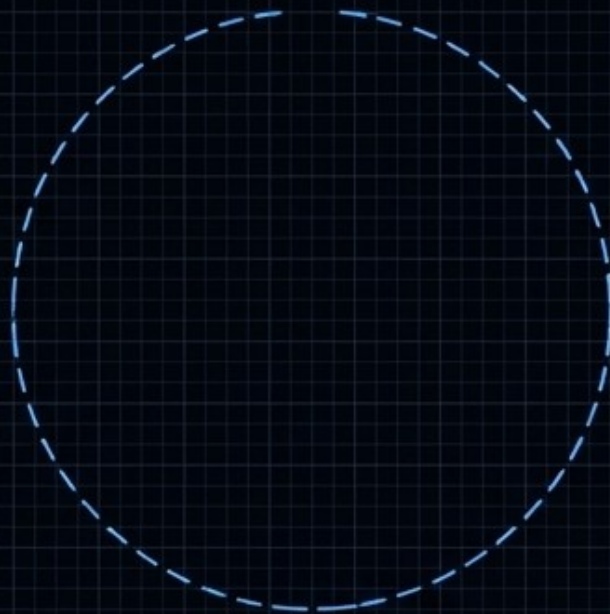


业界的误区



编写复杂的流程图调度模型。
真正做决策的是代码，不是 Intelligence。

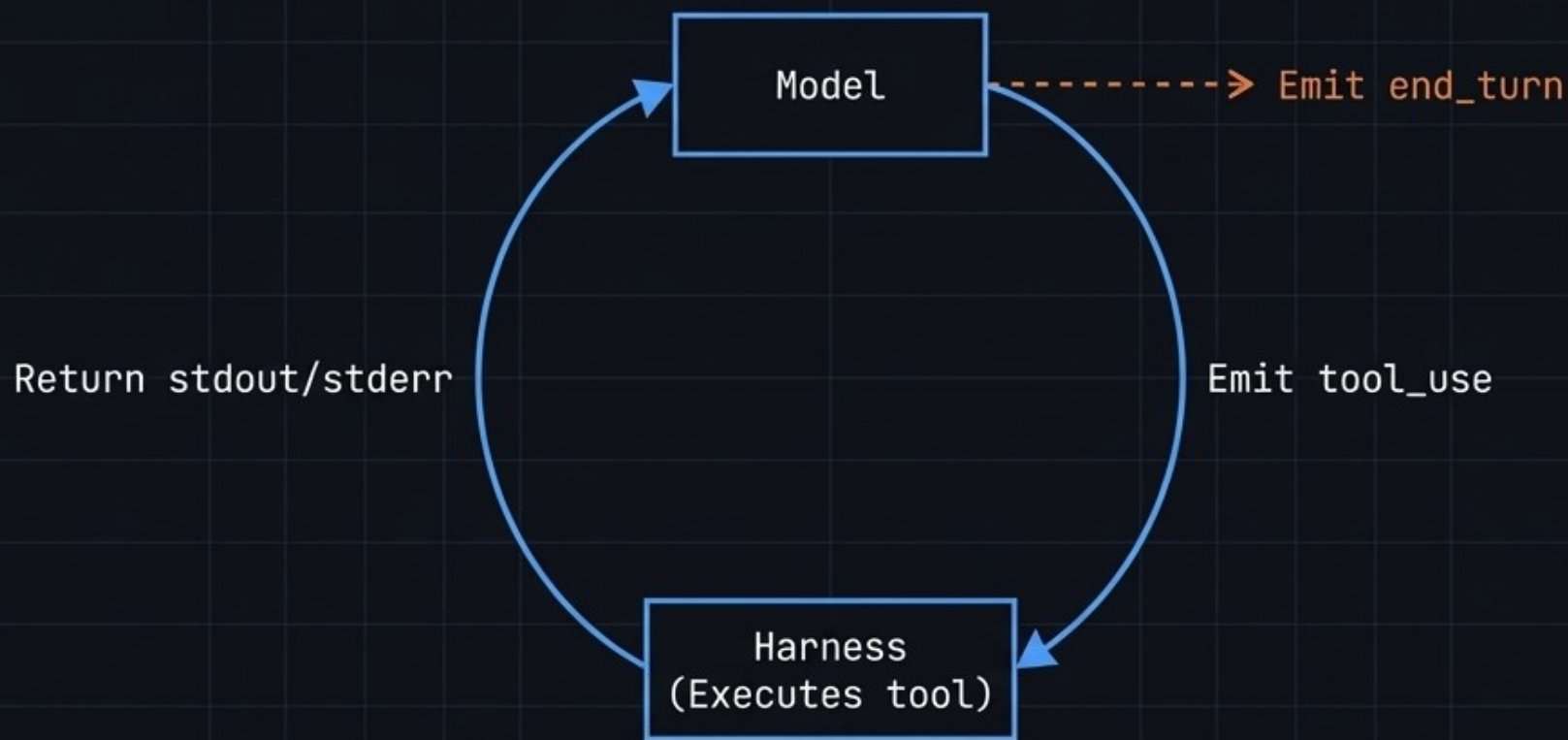
真正的 Paradigm Shift



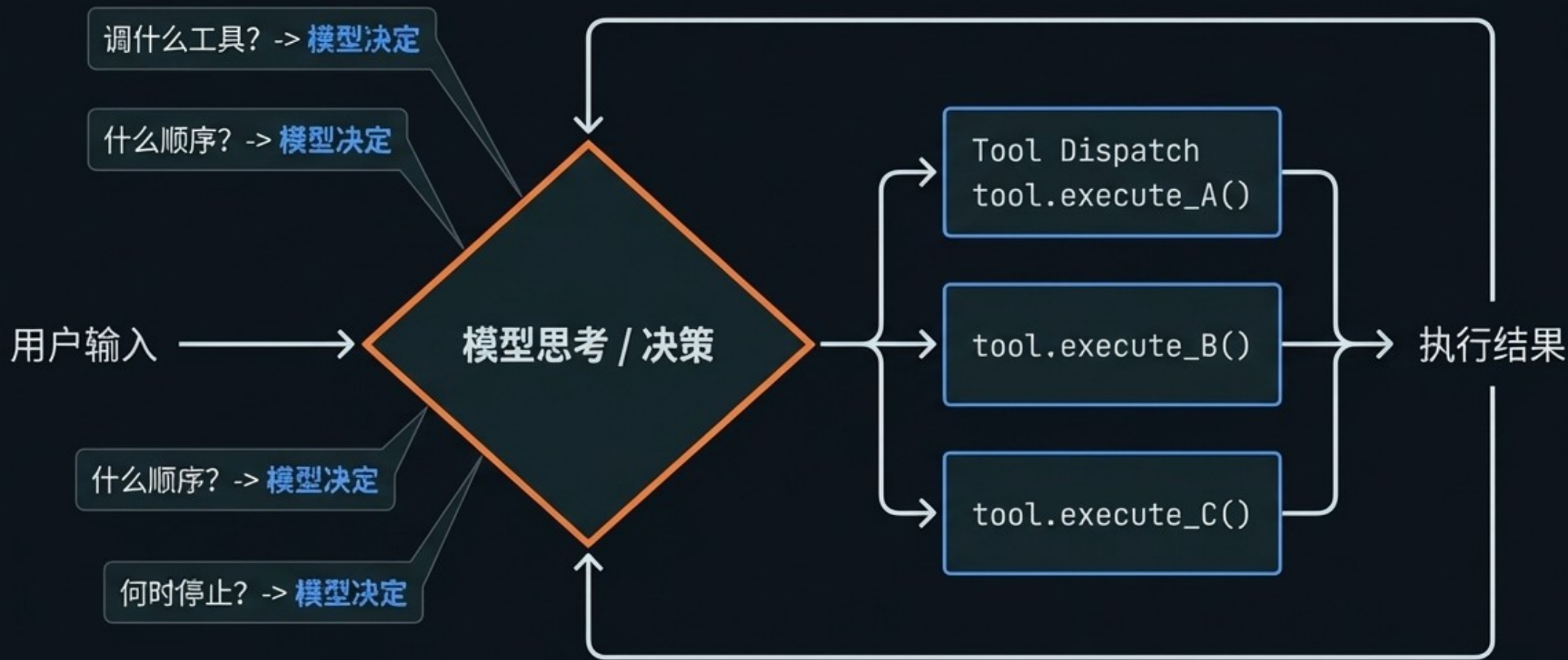
为 Intelligence 提供一个原生工作环境。

模型就是 Agent，代码只是 Harness。

Claude Code 的心脏：永远不变的 30 行循环



所有的复杂机制（TodoWrite、Subagents、三层压缩）都叠加在这个永远不变的循环上。Harness 只做三件事：跑循环、执行工具、喂回结果。



约 30 行代码。这个心脏循环永远不变。Harness 只做三件事：跑循环、执行工具、喂回结果。扩展能力不等于修改内核。

痛点 1: Context Fade (失忆)

解法: TodoWrite 约束

- 不是替模型做计划，是逼它写下来。
- 约束规则：最多20项，仅1个 in_progress，连续3轮未更新系统自动提醒。

痛点 2: Context Pollution (探索垃圾)

解法: Subagent 隔离

- 主进程写代码，子进程读文件探索。
- 完全隔离，保护主进程认知清晰度。

痛点 3: 知识瓶颈

解法: Skills 渐进式加载

- 100 个预置 Skill (如 MCP 规范)，仅按需加载 2-3 个，实现零闲置 Token 消耗。

痛点 4: 上下文爆表

解法: 三层压缩防御

- Micro (旧工具结果变占位符) -> Auto (超50k 自动摘要) -> Manual (主动调用 compact)。
- 持久化靠磁盘，Context 是一次性的。

防遗忘 (Context Fade)

TodoWrite

最多 20 项，逼迫模型将隐式计划转化为显式文件。

约束即脚手架。

防污染 (Context Pollution)

Subagent

探索任务（读大量文件）交给子进程。主线上下文免受探索垃圾污染。

进程隔离 = 认知隔离。

防过载 (Context Overload)

Skills & Compaction

知识按需加载。超 50k Token 触发摘要，旧结果替换为占位符。

磁盘是持久的，内存是一次性的。

TEST: SWE-bench | MODEL: Claude Opus 4.5

42% → 78%

仅仅因为换了一套 Harness.

Claude Code 的根本局限：它是被动的。

用户输入



用户输入



用户说话，它干活。用户不说话，它就闲着。

永远等人踹一脚才动。

用户说话，它干活。
用户不说话，它就闲着。

它永远等人踹一脚才动。

活跃



无尽的被动等待...

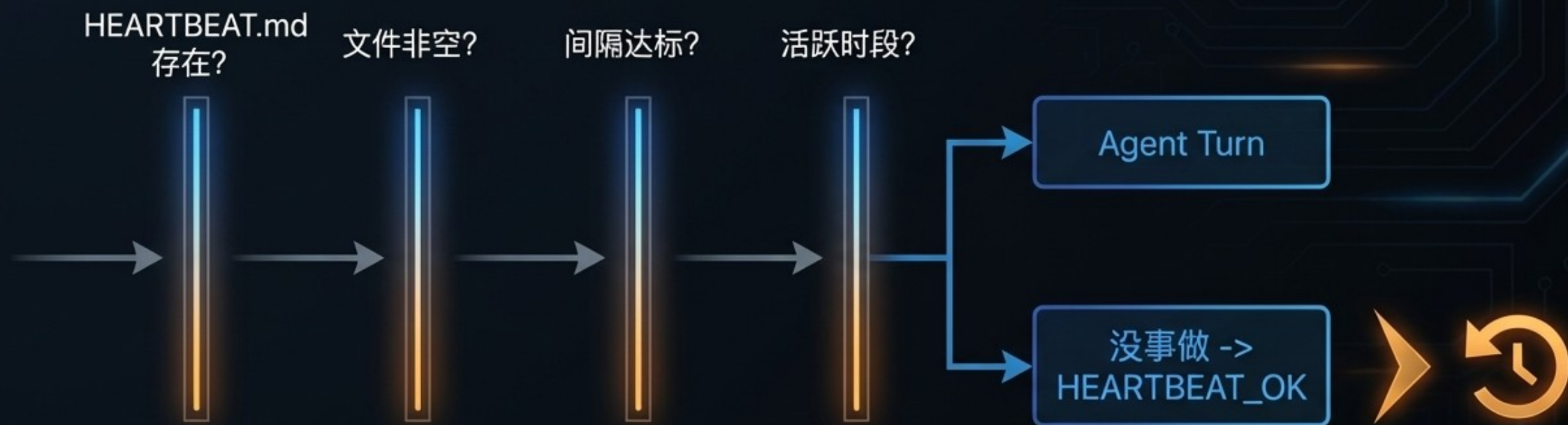
PART 2: THE PROACTIVE SHIFT

如何让 Agent 自己找事干？

从被动等待，到主动驻留。

核心引擎：零污染的心跳自检

后台定时问 Agent：“你现在有事要干吗？”



如果没事做，静默回滚当前轮次的的上下文与时间戳。就好像这一轮从未发生过。不浪费一次 API 调用，不污染一丝上下文。

Heartbeat 机制：没事就沉默



后台定时器周期性询问 '你现在有事干吗?' ——绝不浪费一次 API 调用。

记忆防污染机制

如果 Agent 醒来后回答 HEARTBEAT_OK (没事干)

机制：截断并回滚这轮 transcript，恢复时间戳。

[User] 你好，今天有什么任务？

[Agent] 我正在检查系统状态...

[System] HEARTBEAT CHECK...

[Agent] HEARTBEAT_OK (No Action)



无信息的轮次，绝不污染上下文。

HEARTBEAT.md 存在?

内容非空?

间隔足够?

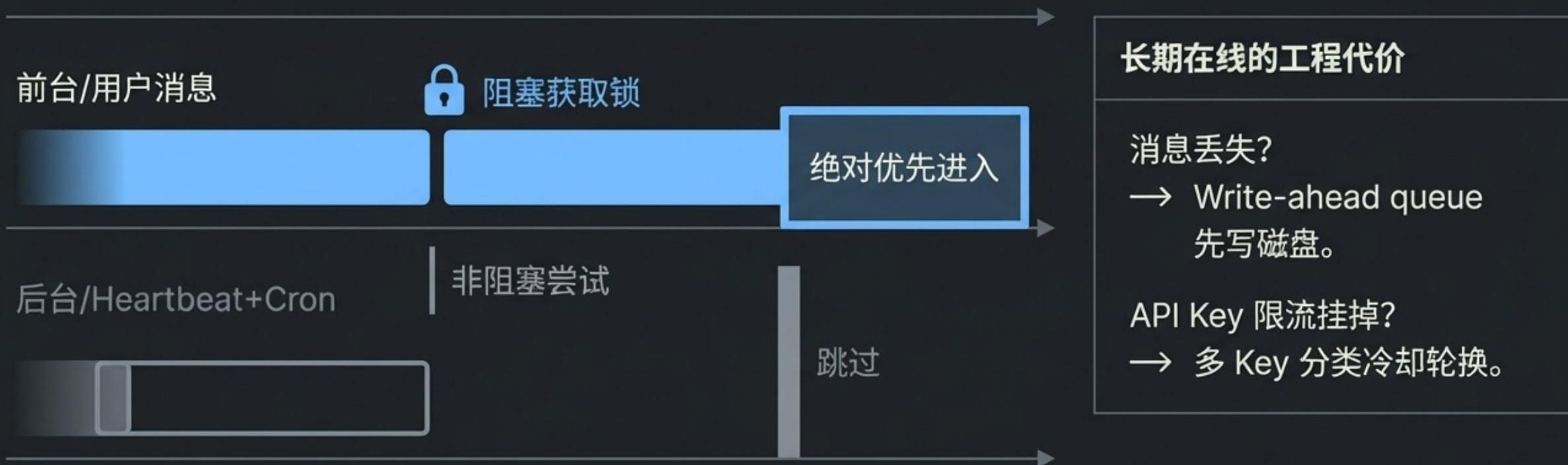
活跃时段?

If Agent replies HEARTBEAT_OK (没事)
-> 状态回滚 (Context Rollback)

恢复时间戳，截断这轮 Transcript。无信息的轮次绝不污染上下文。没事就沉默。

If all pass -> 发起询问: "你现在有事要干吗?"

Gateway 驱动的 Agent Runtime

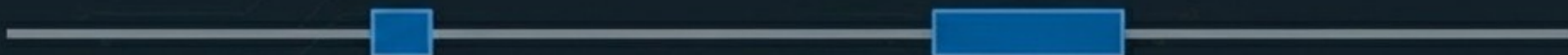


后台永远不中断前台。

Claude Code (Main-session)



OpenClaw (Gateway Runtime)

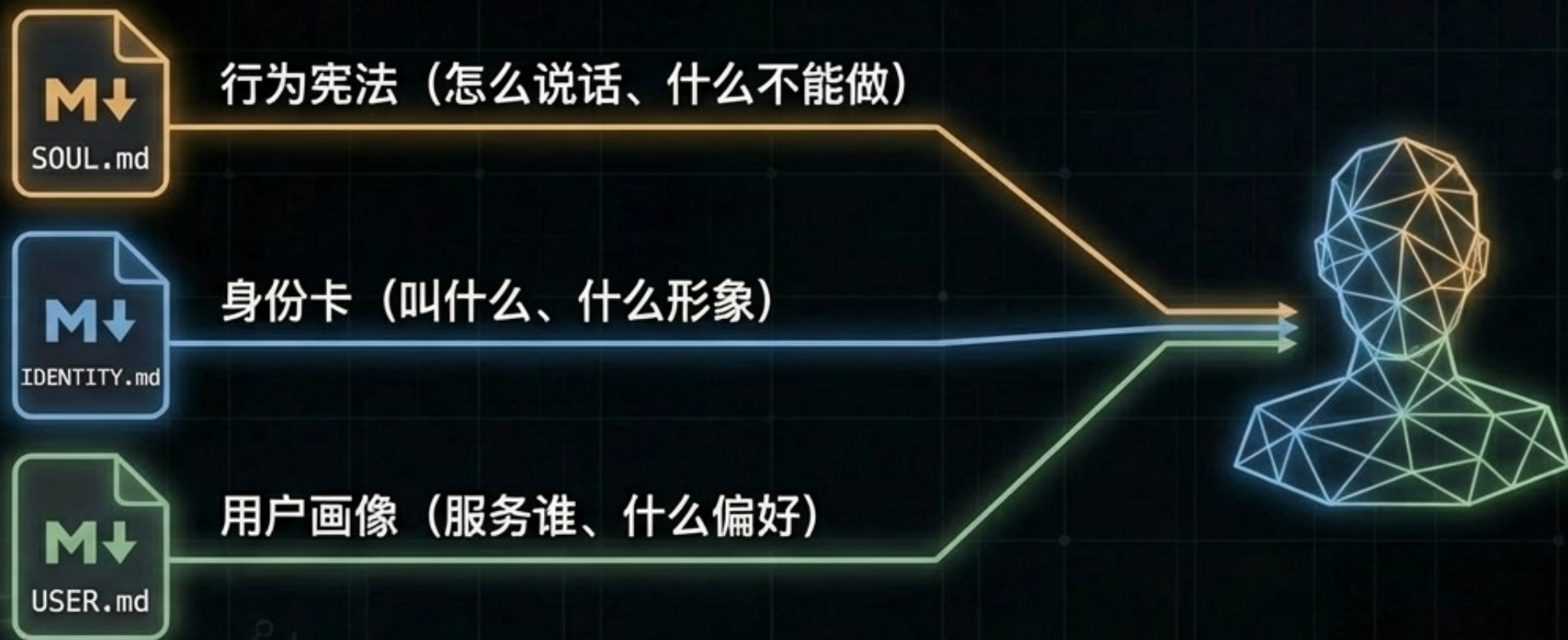
A User inputs
(阻塞获取锁)

B Cron triggers



后台永远不中断前台：非阻塞 Heartbeat 独立运行。

身份与记忆沉淀于文件系统



改文件即改人格，无需改代码。

1. 身份 (Identity)



SOUL.md



IDENTITY.md



USER.md

行为宪法 + 人格 + 偏好。
改文件就改人格，不改代码。

2. 记忆 (Memory)



MEMORY.md



2023-10-25.md

自动注入长期记忆 + TF-IDF 日志检索。
记忆的 Source of Truth 是文件系统，
而非模型的隐式 Context。

3. 引擎 (Heartbeat)



“你现在有事要干吗？”

一个后台定时器，周期性主动向 Agent 提问。

维度	Claude Code	OpenClaw
触发模式	被动等待	用户 + Heartbeat + Cron 主动自检
在线周期	单次对话	长周期驻留
架构中枢	无	Gateway 调度器

Agent 终于“活”了起来。

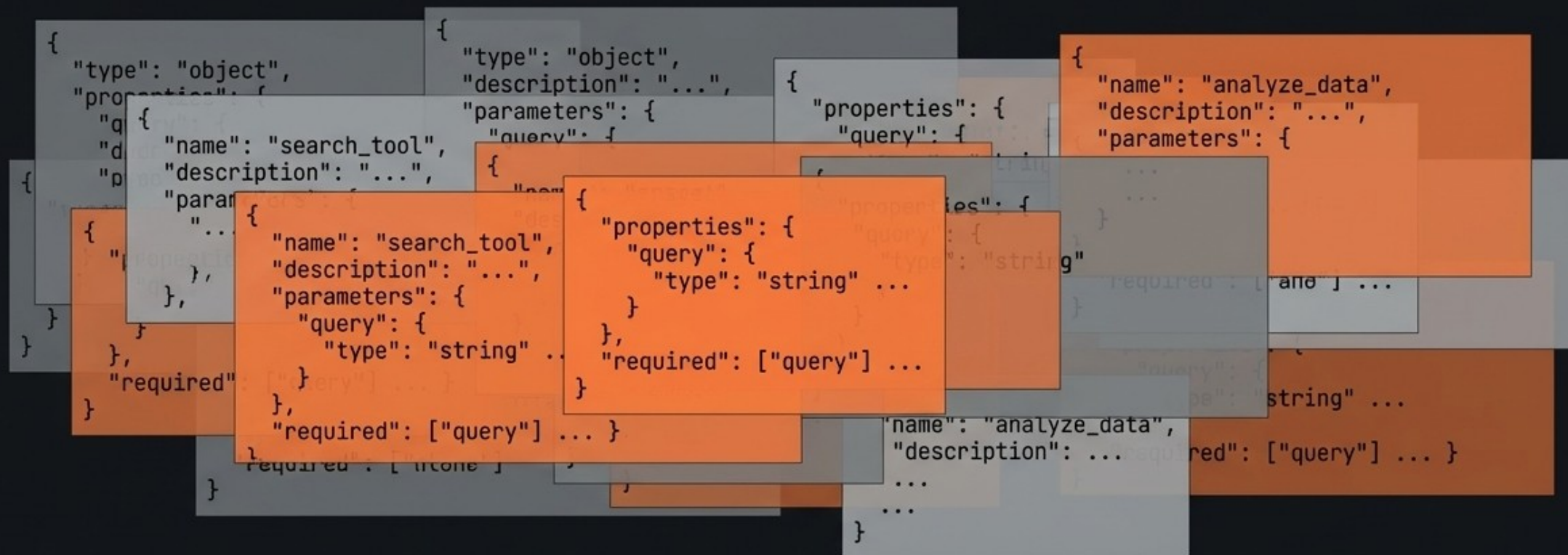
但它们都困在同一个牢笼里

JSON Function Calling.

[illegible]

- 每次交互都是一次离散的函数调用。
- 离散工具，不可组合。
- 50 个工具的 Schema 描述，可能就要吃掉 50,000 Tokens。

当前 Agent 世界的歧途：JSON Function Calling



每次交互都是离散的函数调用。工具不可组合。
50 个工具的 Schema 描述就能占满 5 万 Token。

为什么 Agent 工具不能就是 CLI?

Agent 世界正在重新发明一套极其臃肿的 JSON 协议。

主流 Agent Harness

```
{
  {
    "tool": "code_interpreter",
    "args": {
      "code": "...",
      "language": "python"
    },
    "description": "...",
    "required": [...]
  },
  {
    "tool": "code_matapot",
    "args": {
      "code": "...",
      "language": "python"
    },
    "description": "...",
    "required": [...]
  }
}

[
  {
    "tool": "code_interpreter",
    "args": {
      "code": "...",
      "language": "python"
    },
    "description": "...",
    "required": [...]
  },
  {
    "tool": "code_interpreter",
    "args": {
      "code": "...",
      "language": "python"
    },
    "description": "...",
    "required": [...]
  }
]
```

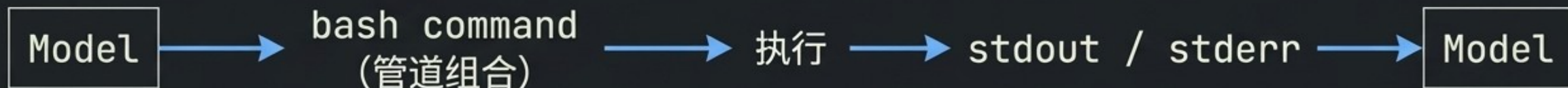
50个工具 = 50,000 tokens

真实世界

—— git —— docker —— grep ——>

Unix 的极简哲学:
Everything is a command

Kode 的核心主张：放弃 JSON，拥抱 Unix



扩展能力不再是定义臃肿的新 JSON Schema，只需增加一个 CLI 命令。

高先验密度

模型预训练看过海量
Linux 操作

天然可组合

`A | B` 或 `A > file`

天然可发现

`--help`

Unix 交互范式的绝对优势



高密度先验知识	天然可组合	天然可发现
模型预训练见过数十亿 Token 的 Linux 操作。它天生知道 <code>grep -r</code> 怎么用，但它不天然懂你自定义的 JSON Schema。	<code>toolA toolB > file</code> JSON 做不到。	遇到未知工具，只需 <code>--help</code> ，无需在 Prompt 里塞满文档。

Context 快满 (100k+ tokens)

```
$ cat > /tasks/situation.md (写入当前态势)  
$ echo > /memory/core.md (写入核心发现)
```

Harness 拦截并清空 Context 快照

新回合重启: "cat /tasks/situation.md 恢复认知"

知识没有丢失，只是从 Context 转移到了文件系统。

知识没有丢失，只是转移到了文件系统

信息不在 Context 里，而在文件系统里。

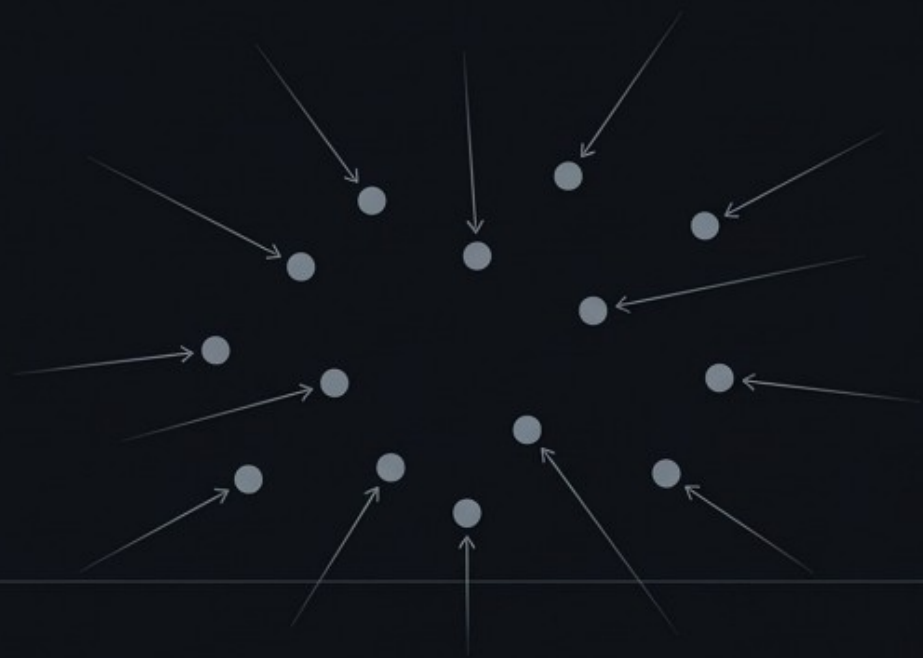
25,000 Tokens

静态全量注入，有效利用率 0.8%

955 Tokens

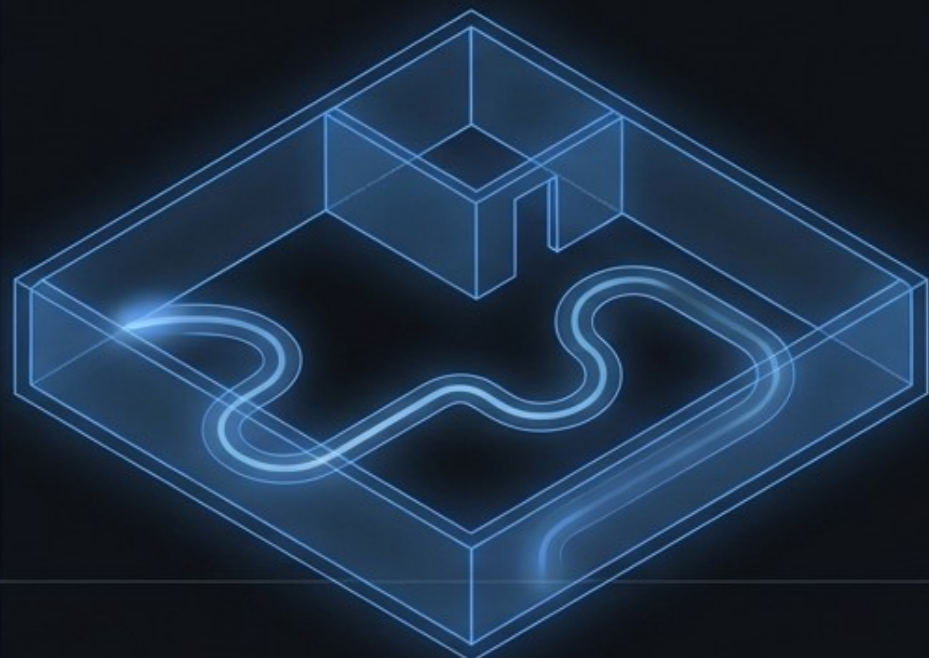
按需 cat 文件，有效利用率 100%

[启动只需一句：'你在 Kode Computer 里。cat /tasks/situation.md 看你的任务。'
Agent 需要什么，就自己 cat 什么。]



工具是被调用的。

离散，无记忆。

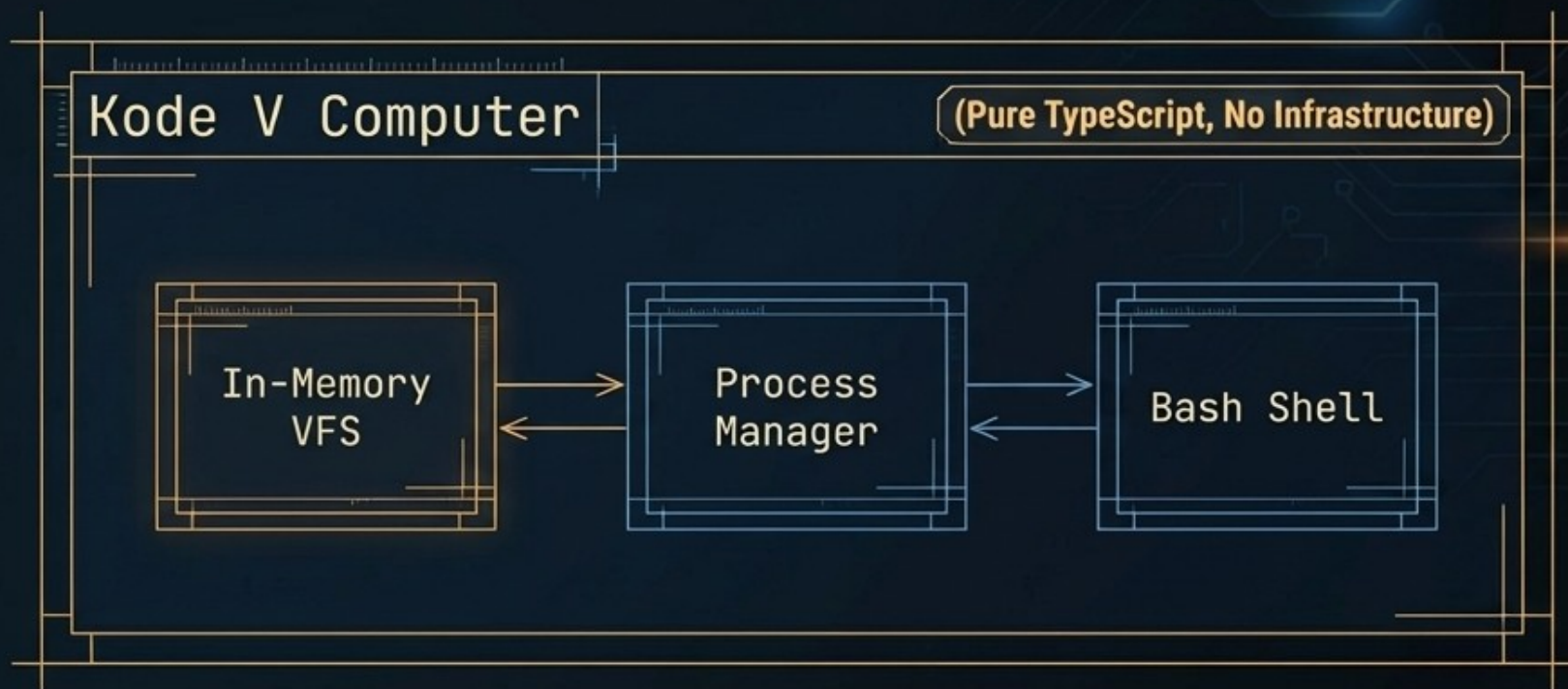


环境是被居住的。

连续，有状态。

环境本身，就是最好的状态管理机制。

Kode V Computer: 下一代 Harness



一台内存中的 Unix-like Agent Computer。不用 JSON，用 bash。

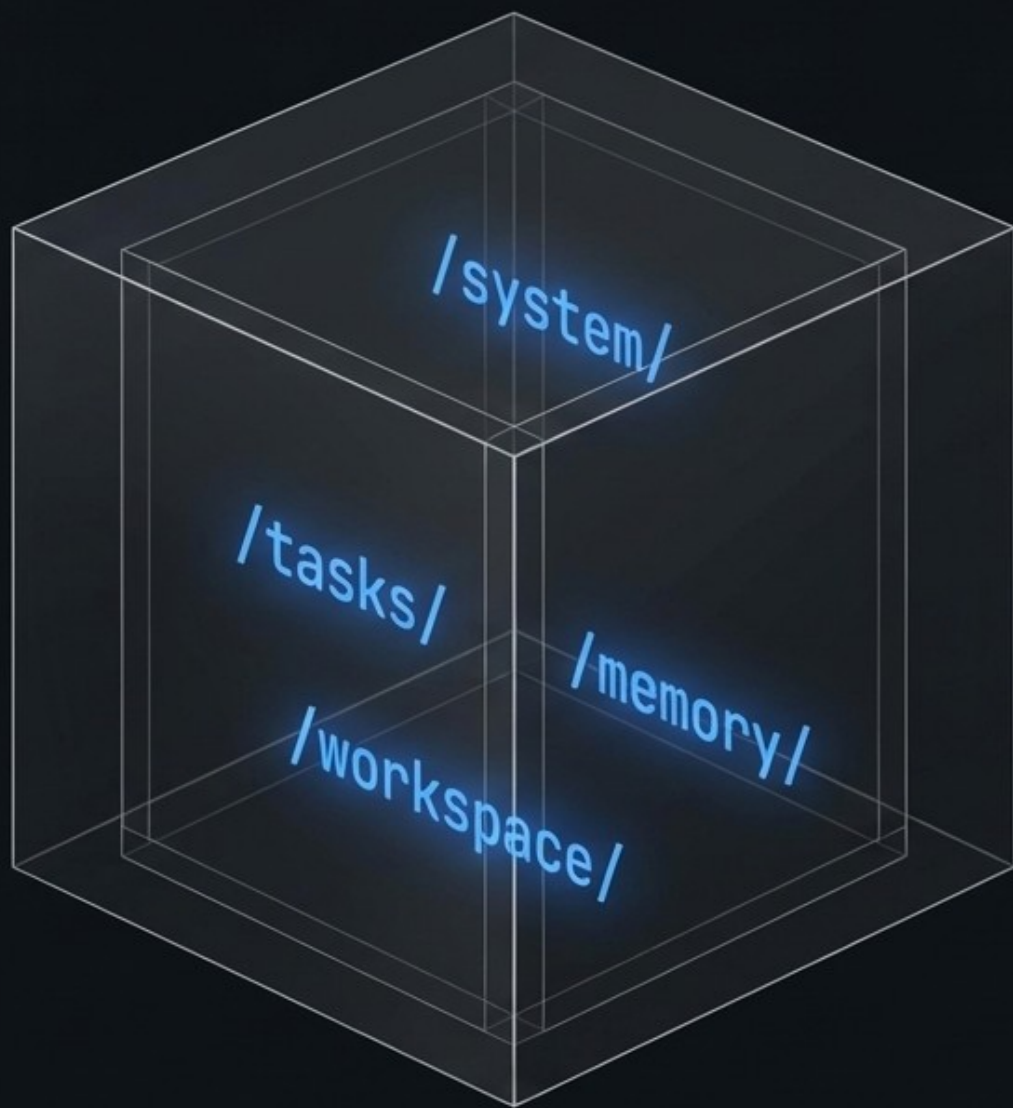
不是 Linux 模拟器，不是 Docker Sandbox。是一个用 Unix 范式构建的纯 TypeScript Agent 运行时界面。

一台内存中的 Unix-like Agent OS

零基础设施（纯 TypeScript，浏览器可跑）

超高先验知识密度（模型天生懂 bash）

天然有状态（文件系统即状态）

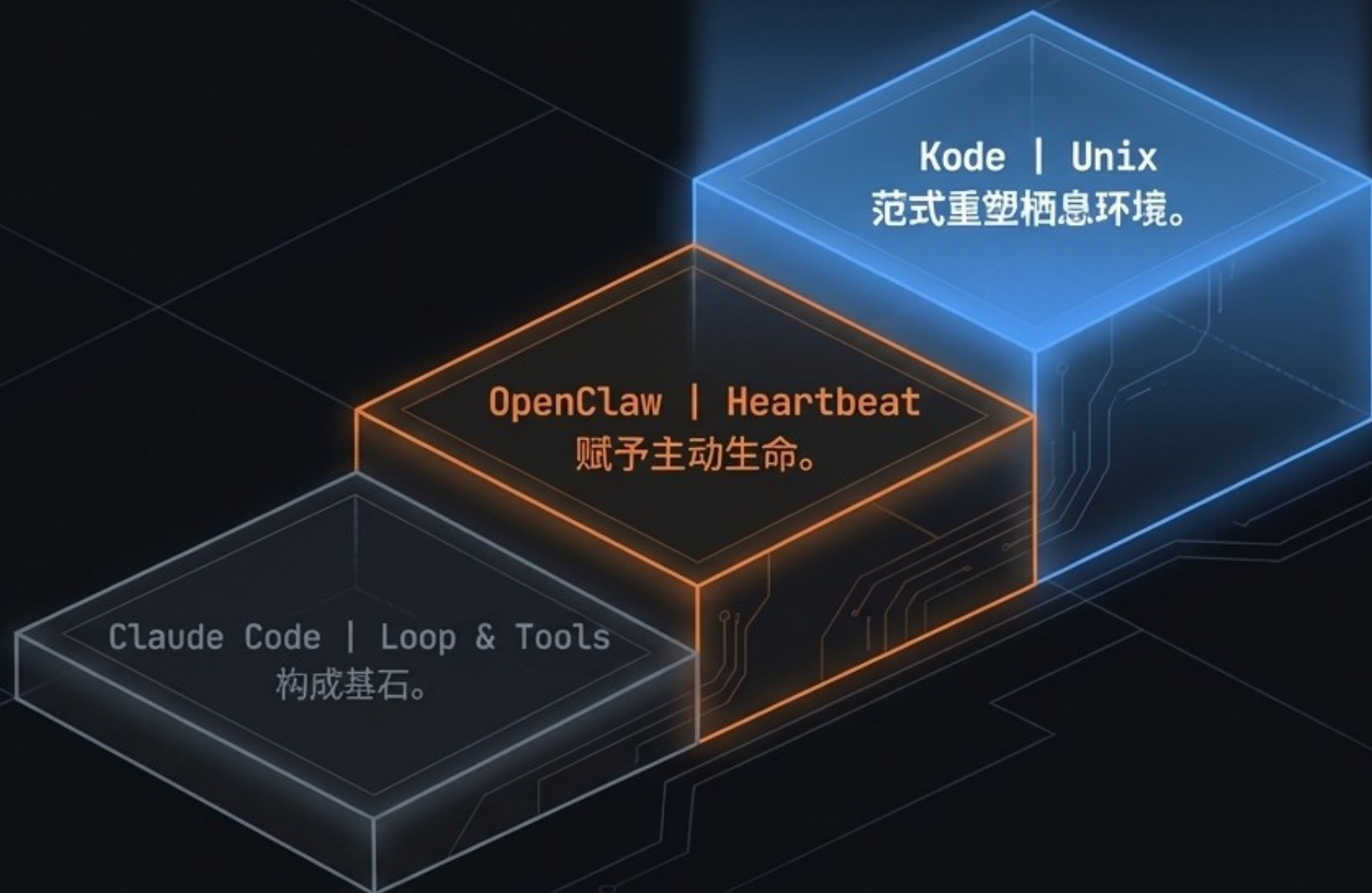


	传统 Agent 环境 (JSON)	Kode 驻留环境 (Unix CLI)
先验知识密度 (Prior Knowledge)	模型需要从零开始学习你自定义的复杂 Schema。	预训练数据中包含数十亿 Token 的 Linux 操作记录。模型天然精通 <code>grep -r</code> 。
可组合性 (Composability)	孤立执行，输出必须回传给大模型再发起下一次调用。	极致的原生组合能力，天然支持管道： <code>tool_A tool_B > output.txt</code> 。
状态管理 (State Management)	极度依赖易碎的 Context Window 来维持状态记忆。	文件系统本身就是最完美的持久化状态。



	Claude Code	OpenClaw	Kode
触发机制	被动响应	主动 (Cron + Heartbeat)	主动在线
动作接口	JSON Function Calling	JSON Function Calling	Native Unix CLI (Bash)
上下文管理	三层压缩防爆	周期回滚防污染	文件系统作为 Progressive Context
核心哲学	循环调度	网关接管	环境即状态 (Habitat)

范式的必然演进



Thanks.

